LAB06 – GRADIENT VÀ PHÁT HIỆN BIÊN

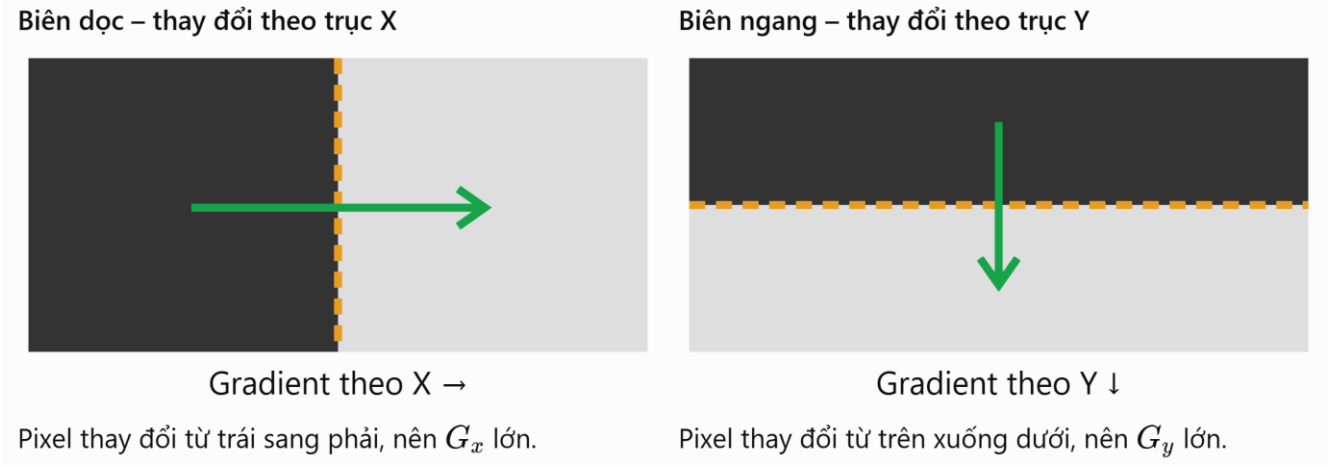
Sobel • Prewitt • Roberts • Laplacian • Canny

1. Mục tiêu và kiến thức kế thừa

- Sử dụng lại convolution2d, padding, Gaussian smoothing và ảnh nhiễu từ LAB05.
- Tính gradient Gx, Gy, magnitude và direction trên array/ảnh thực tế.
- Cài đặt và so sánh Sobel, Prewitt, Roberts, Laplacian trên cùng ảnh.
- Phân tích độ dày cạnh, độ nhạy nhiễu, hướng cạnh và ảnh hưởng threshold.
- Giải thích và mô phỏng Gaussian → Gradient → NMS → Double Threshold → Hysteresis của Canny.

Thư viện: cv2, numpy, matplotlib. Ảnh so sánh dùng chung nguồn và điều kiện tiền xử lý. Dùng CV_32F khi tính gradient/Laplacian để giữ giá trị âm. Các phép threshold so sánh cần nêu rõ chuẩn hóa và ngưỡng.

2. Bài tập



Bài 1. Tính Sobel 3×3 trên array – Gradient

Mục tiêu: Tính gradient theo hai trục X và Y, xác định độ lớn (magnitude) và hướng (direction) của gradient. Thực hiện tính tay trên ma trận 5×5 và đối chiếu kết quả bằng Python.

Cho ảnh xám A kích thước 5×5, có sự thay đổi mức xám từ 10 sang 200 theo cả chiều trái sang phải và trên xuống dưới:

10	10	10	10	10
10	10	10	10	10
10	10	10	200	200
10	10	200	200	200
10	10	200	200	200

Kernel K_x

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Phát hiện biên đứng.

Kernel K_y

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Phát hiện biên ngang.

Độ lớn và hướng gradient

$$G = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \text{atan2}(G_y, G_x)$$

1. Tính Gx và Gy.

2. Tính magnitude sqrt(Gx^2+Gy^2).

3. Tính tay một vị trí cạnh và đối chiếu code.

Code skeleton (TODO):

```
A = np.array([...],dtype=np.float32)
Kx = np.array([[-1,0,1],[-2,0,2],[-1,0,1]],np.float32)
# TODO: Ky, Gx, Gy, magnitude, phép tính 1 pixel
```

Sản phẩm: Bảng array và phép tính tay.

NHẬN XÉT BẮT BUỘC: |Gx| và |Gy| phản ánh thay đổi theo trục nào?

Bài 2. Sobel X/Y trên ảnh thật

Quan sát đạo hàm theo x/y, magnitude và ảnh hưởng ksize.

1. Đọc grayscale; cv2.Sobel(CV_32F) với ksize=3,5.

2. Hiển thị signed Gx/Gy bằng chuẩn hóa có tâm 0; hiển thị |Gx|, |Gy| và magnitude.

3. So sánh cạnh đứng/ngang và ảnh hưởng ksize.

Code skeleton (TODO):

```
# TODO: gx=cv2.Sobel(gray,cv2.CV_32F,1,0,ksize=3)
# TODO: gy, magnitude, normalize, plot
```

Sản phẩm: Figure Original | Gx | Gy | Magnitude.

NHẬN XÉT BẮT BUỘC: Vì sao chuyển thẳng sang uint8 có thể làm mất gradient âm?

Bài 3. Hướng gradient

Phân tích hướng biến thiên cường độ.

1. Tính angle = arctan2(Gy,Gx) và chuyển độ.

2. Hiển thị bản đồ góc trên các pixel có magnitude vượt ngưỡng.

3. Chọn 5 pixel cạnh, báo cáo Gx/Gy/magnitude/angle.

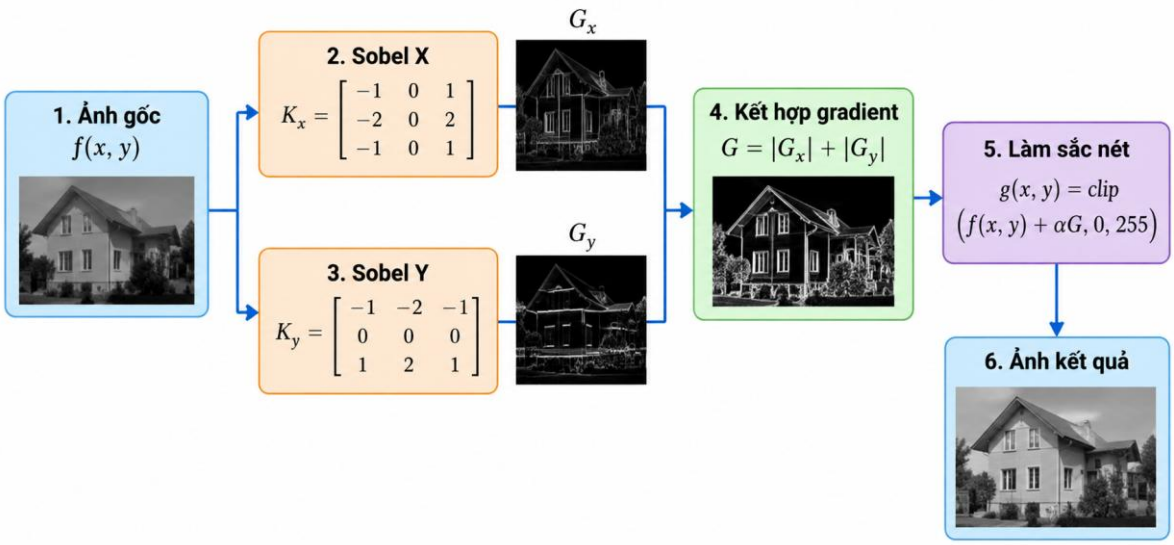
Code skeleton (TODO):

```
# TODO: angle=np.degrees(np.arctan2(gy,gx))
# TODO: mask = magnitude > threshold
# TODO: visualize và thống kê 5 pixel
```

Sản phẩm: Bản đồ magnitude/direction và bảng mẫu.

NHẬN XÉT BẮT BUỘC: Hướng gradient song song hay vuông góc đường biên?

Bài 4. Pipeline làm sắc nét ảnh bằng Sobel



Bài 5. Prewitt

Tự định nghĩa kernel Prewitt và so sánh Sobel.

1. Khai báo Kx/Ky Prewitt; tính với filter2D hoặc convolution2d (đúng quy ước dấu).

2. Tính magnitude và so sánh Sobel trên cùng ảnh/scale.

3. Thử ảnh có nhiễu từ LAB05.

$$H_x = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

$$H_y = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

- Bước 1: Tính $G_x = I * H_x$ và $G_y = I * H_y$
- Bước 2: Tính $G_x + G_y$

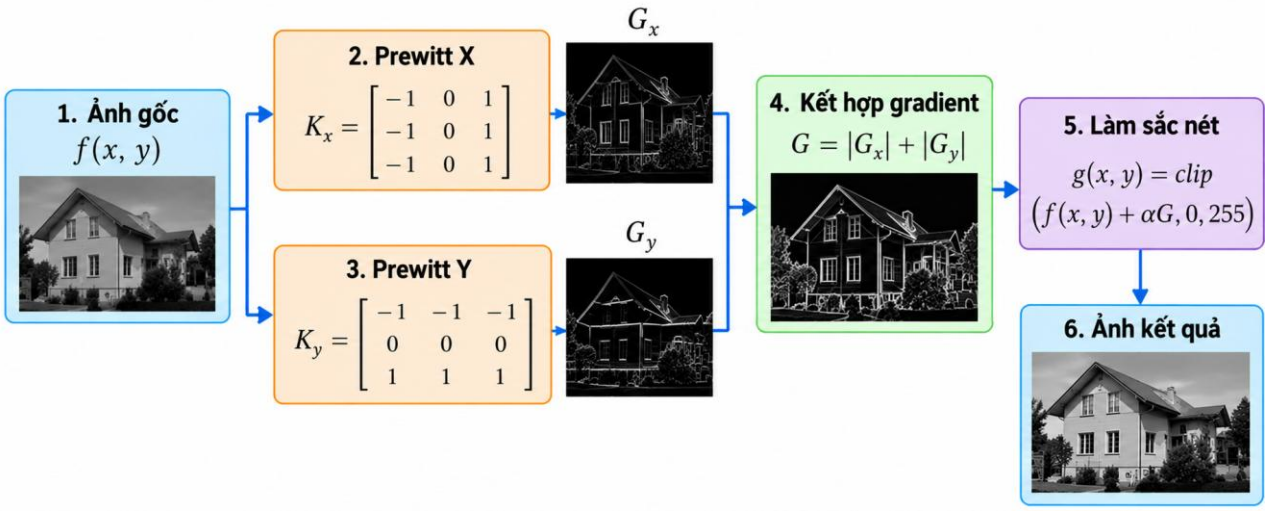
Code skeleton (TODO):

```
Kx=np.array([[-1,0,1],[-1,0,1],[-1,0,1]],np.float32)
# TODO: Ky, gx, gy, magnitude, compare
```

Sản phẩm: Figure Sobel/Prewitt và phân tích.

NHẬN XÉT BẮT BUỘC: Trọng số hàng giữa của Sobel ảnh hưởng như thế nào?

Bài 6. Pipeline Prewitt Sharpening



Bài 7. Roberts Cross

Khảo sát hai kernel 2×2 và sự nhạy nhiễu.

1. Dùng hai kernel Roberts; chỉ rõ anchor mặc định với kernel chẵn và xử lý biên.

2. Tính hai đáp ứng và magnitude trên float32.

3. So sánh với Sobel/Prewitt ở vùng chi tiết nhỏ.

```
# Kernel Robert
kx = np.array([[-1, 0],
               [ 0, 1]], dtype=np.float32)

ky = np.array([[ 0, -1],
               [ 1,  0]], dtype=np.float32)

# Áp dụng filter2D
gx = cv2.filter2D(img, cv2.CV_32F, kx)
gy = cv2.filter2D(img, cv2.CV_32F, ky)

# Magnitude
mag = np.sqrt(gx**2 + gy**2)
mag = np.uint8((mag / mag.max()) * 255)
```

Code skeleton (TODO):

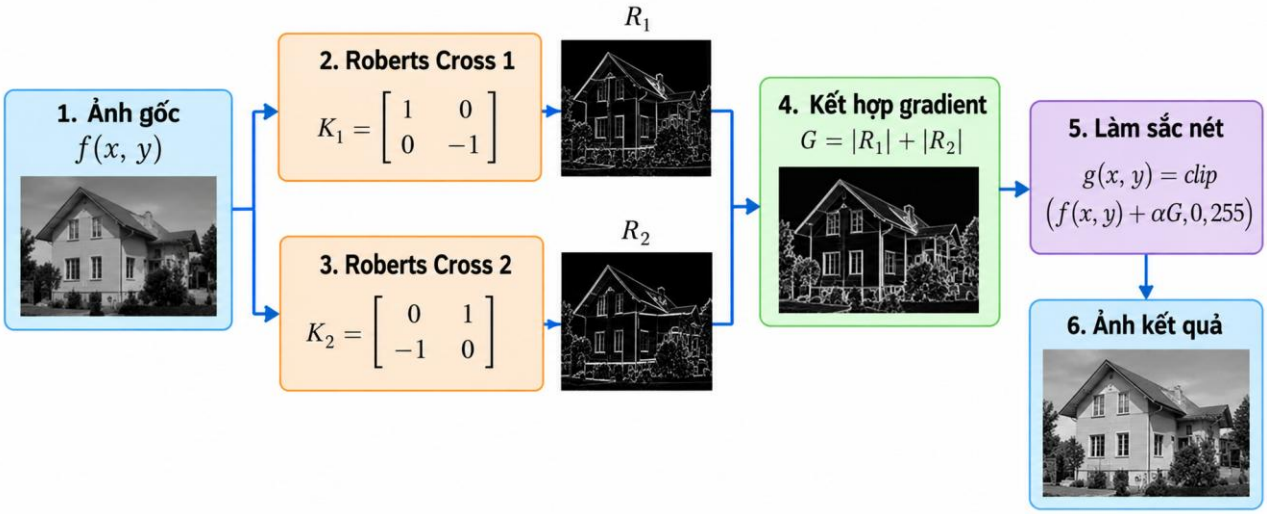
```
K1=np.array([[1,0],[0,-1]],np.float32)
K2=np.array([[0,1],[-1,0]],np.float32)
# TODO: filter2D(CV_32F), magnitude, visualize
```

Sản phẩm: Ảnh Roberts và vùng crop đối chiếu.

NHẬN XÉT BẮT BUỘC: Vì sao kernel nhỏ có thể phản ứng mạnh với nhiễu?

Bài 8. Roberts Cross Sharpening

Áp dụng hai kernel Roberts Cross 2×2 để tính hai đáp ứng gradient, kết hợp bằng chuẩn L1 (hoặc L2), sau đó cộng độ lớn gradient với ảnh gốc nhằm tăng cường vùng biên.



Bài 9. Laplacian và tiền xử lý Gaussian

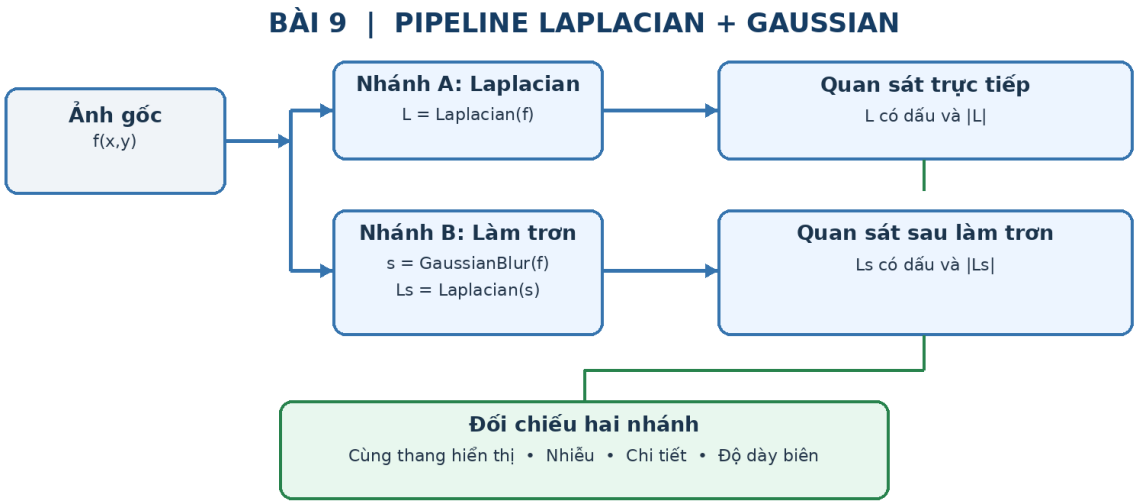
Dùng đạo hàm bậc hai để quan sát cạnh và nhiễu.

1. Tính cv2.Laplacian(CV_32F), hiển thị ảnh signed và |response|.

2. GaussianBlur trước Laplacian; so sánh với ảnh không smoothing.

3. Liên hệ Laplacian sharpening ở LAB05: cùng toán tử nhưng mục đích/đầu ra khác.

Laplacian trước và sau Gaussian Blur



Quy trình: Ảnh gốc → (A) Laplacian trực tiếp; (B) Gaussian Blur → Laplacian → hiển thị ảnh có dấu và trị tuyệt đối → so sánh trên cùng thang cường độ.

Lưu ý: Đây là pipeline khảo sát đáp ứng đạo hàm bậc hai, không phải pipeline làm sắc nét. Nếu mở rộng sang sharpening với kernel Laplacian có tâm âm, dùng g = clip(f − αL, 0, 255); nếu đảo dấu kernel, phải đảo dấu phép cộng/trừ tương ứng.

Code gợi ý: f = gray.astype(np.float32); L = cv2.Laplacian(f, cv2.CV_32F, ksize=3); s = cv2.GaussianBlur(f, (5, 5), 1); Ls = cv2.Laplacian(s, cv2.CV_32F, ksize=3).

Code skeleton (TODO):

```
# TODO: lap=cv2.Laplacian(gray,cv2.CV_32F)
# TODO: smooth=cv2.GaussianBlur(gray,(5,5),1)
# TODO: lap_smooth, compare
```

Sản phẩm: Figure Laplacian trước/sau blur.

NHẬN XÉT BẮT BUỘC: Vì sao Laplacian nhạy nhiễu? Dấu Laplacian có ý nghĩa gì?

Bài 10. Benchmark bốn toán tử và threshold

So sánh công bằng Sobel, Prewitt, Roberts, Laplacian.

1. Trên cùng ảnh sạch và nhiễu, tính response từng toán tử.

2. Chuẩn hóa magnitude/|Laplacian| theo quy tắc công bố rõ; thử ít nhất 3 threshold.

3. Tạo bảng phân tích 4 tiêu chí: độ dày cạnh, nhiễu, hướng cạnh, threshold.

Code skeleton (TODO):

```
# TODO: responses={Sobel,Prewitt,Roberts,Laplacian}
# TODO: normalize / threshold theo quy tắc chung
# TODO: plot 3 threshold + hoàn thiện bảng
```

Sản phẩm: Bảng 4×4 và hình minh họa ngưỡng.

NHẬN XÉT BẮT BUỘC: Cạnh mảnh/dày, nhiễu, hướng và threshold thay đổi ra sao? Tránh kết luận chung chung.

Bảng nhận xét bắt buộc – Bài 10

Phương pháp	Độ dày cạnh	Nhạy nhiễu	Hướng cạnh	Threshold
Sobel	……	……	……	……
Prewitt	……	……	……	……
Roberts	……	……	……	……
Laplacian	……	……	……	……

Bài 11. Ảnh hưởng của nhiễu và smoothing

Tái sử dụng Gaussian Noise và Salt & Pepper Noise từ LAB05.

1. Áp dụng Sobel và Canny trực tiếp trên ảnh nhiễu.

2. Thử GaussianBlur ít nhất hai cấu hình và Median khi nhiễu xung.

3. Đối chiếu số cạnh giả, cạnh yếu bị mất và mức giữ chi tiết.

Code skeleton (TODO):

```
# TODO: noisy_gaussian/noisy_sp từ LAB05
# TODO: edge_noisy, edge_denoised
# TODO: thử 2 mức blur; plot
```

Sản phẩm: Figure noisy/denoised/edge và phân tích.

NHẬN XÉT BẮT BUỘC: Blur mạnh làm giảm cạnh giả nhưng làm mất cạnh yếu ra sao?

Bài 12. Canny từng bước

Tìm hiểu pipeline và đối chiếu cv2.Canny.

1. Gaussian smoothing → Sobel Gx/Gy → magnitude/direction.

2. Tự cài NMS: lượng tử hóa góc 0/45/90/135°, giữ cực đại cục bộ.

3. Double threshold phân loại strong/weak/non-edge; hysteresis kết nối 8-neighbor.

4. Đối chiếu cv2.Canny trên cùng ảnh và nêu khác biệt do quy ước gradient/threshold.

Code skeleton (TODO):

```
def non_maximum_suppression(magnitude, angle):
    # TODO: 4 hướng; giữ cực đại cục bộ
    pass
def double_threshold(nms, low, high):
    # TODO: strong, weak, non-edge
    pass
def hysteresis(strong, weak):
    # TODO: lan truyền 8-neighbor từ strong
    pass
# TODO: so sánh cv2.Canny(...)
```

Sản phẩm: Ảnh từng bước và bảng so sánh Canny OpenCV.

NHẬN XÉT BẮT BUỘC: NMS làm mảnh cạnh thế nào? Hai ngưỡng và hysteresis giữ/bỏ cạnh yếu ra sao?

Bài 13. Challenge – Pipeline phát hiện biên thực tế

Ứng dụng vào tài liệu, sản phẩm hoặc bề mặt.

1. Xây dựng grayscale → denoise phù hợp → edge detector → hậu xử lý nếu cần.

2. Thử ≥3 cấu hình blur và low/high threshold; hiển thị kết quả trung gian.

3. Phân tích ít nhất 1 trường hợp thất bại do nhiễu, nền phức tạp hoặc cạnh yếu.

Code skeleton (TODO):

```
def edge_pipeline(image, blur_ksize=5, low=50, high=150):
    # TODO: grayscale, denoise, Canny, optional morphology
    # TODO: return các bước trung gian
    pass
```

Sản phẩm: Pipeline chạy được, hình minh họa và nhận xét.

NHẬN XÉT BẮT BUỘC: Khi thay đổi điều kiện chụp, cấu hình nào kém ổn định và vì sao?

4. Câu hỏi củng cố

1. Gradient magnitude và direction khác nhau ra sao?
2. Vì sao Sobel và Prewitt cho kết quả khác nhau?
3. Kernel Roberts 2×2 có hạn chế gì?
4. Laplacian khác gradient bậc nhất thế nào?
5. NMS và hysteresis có vai trò gì trong Canny?
6. Tại sao tăng threshold không phải giải pháp duy nhất để giảm nhiễu?